15 | Working with JSON and other formats

15.1 Parsing JSON

Nowadays JSON is one of the most popular data-interchange formats. For example, very likely you'll have to deal with HTTP requests returning a json-encoded string . Flutter has an incredible support for this format as it allows you to convert objects from and to JSON very quickly.

```
{
    "id": 1,
    "name": "Alberto",
}
```

What's inside curly brackets is called **json object** and it's always made up of a string, which is the key of the field, and a value which can be a string, a number, a boolean, a list, null or another object. This is another example with a JSON array (or list):

```
[
    {
        "id": 1,
        "name": "Alberto",
    },
    {
        "id": 2,
        "name": "Patrick",
    }
]
```

Lists are simply collections of objects. In Flutter you can decide between manually parsing strings,

using facilities from `import "dart:convert"`, or using automatic code generation, which does most of the work automatically. Both ways are fine but you'll see that code generation simplifies maintenance a lot, especially for large JSON objects.

15.1.1 Manual parsing

In all those cases where there's the need to deal with a list or small objects, parsing and maintaining JSON manually is fine. The best approach to handle the received data is the creation of a *model* class from its JSON representation. In other words, you should convert a json-encoded string into a Dart `class`. Let's say we have to parse this string.

```
{ "id": 1, "value": "json test" }
```

First of all there's the need to use the `jsonDecode` method, from `"dart:convert"`, which translates a json string into a Dart object whose type is `Map<String, dynamic>`. As we told you back in chapter 2, this is one of the rare cases in which `dynamic` has to be used.

```
import "dart:convert";

void main () {
    final jsonString = '{ "id": 1, "name": "Alberto" }';

    Map<String, dynamic> data = jsonDecode(jsonString);
}
```

If you think about it, this design choice really makes sense because a json object is nothing more than a map whose keys are strings and values can be integers, strings, objects or lists. There's the possibility to make a 1:1 comparison which is very expressive:

JSON string

```
{
    "id": 1,
    "name": "Alberto"
}
```

Dart object

```
Map<String, dynamic> data = {
    "id": 1,
    "name": "Alberto"
}
```

The values of the map are `dynamic` because we have to take care of converting the content into a proper data type. The next step is the creation of a **model** class which converts a given object from and to JSON.

```
class Person {
    final int id;
    final String value;

    // 1.
    Person._({
        required this.id,
        required this.value
    });

    // 2.
    factory Person.fromJson(Map<String, dynamic> json) =>
        Person._(
            id: json["id"],
            value: json["name"],
        );

    // 3.
    Map<String, dynamic> toJson() =>
        {
            "id": id,
            "name": value
        };

    @override
    String toString() => "$id - $value";

}
```

All the parsing logic lies inside `class Person` and nowhere else; casts only happen internally so from the outside there's no need to deal with `dynamic`. In practice, calling `Person.fromJson()` is safe because the developer is not asked to make any cast since they're hidden internally.

1. The constructor is private because this class can be instantiated only if a well-formed json string is passed via `factory`.

2. The `factory` constructor takes the result of a `jsonDecode` method and constructs a `Person` object. This is the only place in which we should pay attention to types. We could also have written this...

```
Person._(
    id: json["id"] as int,
    value: json["value"] as String,
);
```

... but the compiler is smart enough to infer the types and perform automatic casts. Note that this approach is required if you set strict implicit casts rules in `analysis_options.yaml`.

3. Converts the object into a data structure that can be passed to `jsonEncode`, which will convert the `Map` into a JSON string. It's very easy to use:

```
final jsonString = jsonEncode(personObject);
```

The `jsonEncode` method automatically calls `Map<String, dynamic> toJson()` so be sure to have it defined otherwise an error will occur.

As you can see, parsing and converting is all around the creation of a **model class** which should define `factory MyObj.fromJson` and `MyObj.toJson`. Thanks to these conversion methods, handling strings is very easy:

```
void main () {
    final jsonString = '{ "id": 1, "name": "Alberto" }';

    final decodedMap = jsonDecode(jsonString);
    final user = Person.fromJson(decodedMap);

    print("${user.id}"); // prints "1"
    print("${user.value}"); // prints "Alberto"

    final json = jsonEncode(user);
    print(json); // prints "{'id': 1, 'name': 'Alberto'}"
}
```

Having a private constructor, a `Person` object can only be created from a JSON string and nothing more. Any model class is asked to, at least, define:

- a `factory` constructor which creates new instances using the returned value of `jsonDecode`;
- a method called `toJson()` that will be automatically called by `jsonEncode`

15.1.1.1 Parsing lists

Very often JSON strings are long lists containing a series of objects and of course Dart has an easy way to deal with them. Suppose you had to work with the following string:

```
[
    { "id": 1, "name": "Alberto" },
    { "id": 2, "name": "Felix" },
    { "id": 3, "name": "Rémi" },
    { "id": 4, "name": "Matt" }
]
```

In order to parse a list there's the need to intuitively use a `List<T>` object and then treating each item as a JSON object. We're using the same model class we've created in the previous section.

```
void main() {
    final jsonString =
        '''[
            { "id": 1, "name": "Alberto" },
            { "id": 2, "name": "Felix" },
            { "id": 3, "name": "Rémi" },
            { "id": 4, "name": "Matt" }
        ]''';

    // It's a list of json objects
    List<dynamic> jsonList = jsonDecode(jsonString);

    // Each object is converted by the model class
    final List<Person> people = jsonList
        .map((value) => Person.fromJson(value))
        .toList();
}
```

The decoder is allowed to return a `List<dynamic>` when it encounters a list; each json object inside it is nicely converted by the model class. A list is simply a collection of objects which can be handled by a model class.

ℹ The usage of a model class is very handy in this case; the code is highly readable

and type safe. If we didn't use `Person.fromJson` there would have been the need to make manual casts for **any** list to be parsed.

A model class is always required to keep your code readable and easily testable.

15.1.1.2 Parsing nested objects

With the manual approach, things start to get complicated when the JSON string is quite complex because, for example, it's made up of nested objects or lists. Imagine having to work with this apparently simple string:

```
{
    "type": "Developers",
    "data": [
        { "id": 1, "name": "Alberto" },
        { "id": 2, "name": "Felix" },
        { "id": 3, "name": "Rémi" },
        { "id": 4, "name": "Matt" }
    ]
}
```

If you're used to work with REST APIs you might have already encountered something like this millions of times. Jumping into the code, parsing logic starts to get more difficult to write and thus harder for a developer to read. Now `Person` is not enough anymore and we need another model class to handle the outer object:

```
// Same as before
class Person { ... }

// New class for the main object containing 'type' and 'data'
class Data {
    final String type;
    final List<Person> people;

    Data._({
        required this.type,
        required this.people
    });
```

```
    factory Data.fromJson(Map<String, dynamic> json) {
        List<dynamic> list = json["data"];
        List<Person> peopleList = list
          .map((personObj) => Person.fromJson(personObj))
          .toList();

        return Data._(
            type: json["type"],
            people: peopleList,
        );
    }

}
```

Performance is not an issue; the problem instead is the fact that for a fairly simple JSON string we've had to create two different model classes. You can always use manual parsing but with very complex and nested objects, you might get in trouble.

> As time goes by the maintenance can also get harder. What if the REST API from which you depend changed the structure? Potentially, you'd have to refactor most of your code. The solution is the usage of code generation.

Every json object requires his dedicated model class. There isn't a strict rule to determine when you should go for the manual approach but that's up to you; when you see that parsing starts to get hard and there's a lot of boilerplate code, consider moving to automatic parsing.

15.1.2 Automatic parsing

When JSON strings are very complex, with many nested objects and lists, it would be better using code generation which handles most of the tedious work automatically. First of all, you need to add a few dependencies to the project making sure to use the latest version of any package:

```
dependencies
    json_annotation: ^3.0.1
    json_serializable: ^3.4.0
```

The automatic way of parsing JSON is nothing new from what you've just seen. The process

is identical to manual parsing with the only difference that model classes are generated by a command line tool rather than by you, the developer. In order to make an efficient comparison between automatic and manual parsing, we're going to use the same JSON string as example.

```
{ "id": 1, "value": "json test" }
```

As usual, there's the need to create the famous model class which takes care of converting from and to JSON. This time the code looks a bit different from the usual:

```
import 'package:json_annotation/json_annotation.dart';

// 1.
part 'person.g.dart';

// 2.
@JsonSerializable()
class Person {
    final int id;
    final String name;
    Person(this.id, this.name);

    // 3.
    factory Person.fromJson(Map<String, dynamic> json)
        => _$PersonFromJson(json);

    Map<String, dynamic> toJson()
        => _$PersonToJson(this);
}
```

With this setup the code generator tool will be able to automatically create the bodies of both `fromJson` and `toJson`; there's no need for you to make casts or converting data. For now, ignore the errors underlined by the IDE because we're removing them soon.

1. The code generator will automatically create a file called `'person.g.dart'`. Thanks to the `part` directive, the `Person` class is able to access private members in the generated file.

2. The annotation is fundamental as it tells the code generator that `Person` needs the JSON serialization logic to be created. It's a "marker" to recognize the classes that needs code generation.

3. The syntax is a bit weird but it's telling the generator to create for us the code of these methods. You already know what they do from the previous section.

The setup is now ready and it's time to invoke the code generator. Open the terminal from your IDE and run this command, making sure to be at root of the project (which should be already there by default):

```
$ flutter pub run build_runner build
```

Wait until it completes and at the end you will see a new file called `person.g.dart` which has just been created. It contains the same logic you would have implemented by hand as it does casts from `dynamic` to the correct type according with the structure of the model class.

> Any change made to the model class requires the command `build_runner` to be run again in order to update the generated class. Doing this repeatedly might be tedious or you could simply forget about it:
>
> ```
> $ flutter pub run build_runner watch
> ```
>
> Using the command with `watch` rather than `build` automatically runs the code generator whenever a change is made to the class.

The IDE doesn't complain anymore about `person.g.dart` not being declared because it's now been generated by the tool. You're going to use `Person` in the **exact same** way we had already seen in the manual parsing section.

```
void main () {
    final jsonString = '{ "id": 1, "name": "Alberto" }';

    final dataMap = jsonDecode(jsonString);
    final user = Person.fromJson(dataMap);

    print("${user.id}"); // prints "1"
    print("${user.name}"); // prints "Alberto"

    final json = jsonEncode(user);
    print(json); // prints "{'id': 1, 'name': 'Alberto'}"
}
```

In such a simple example it's hard to see why code generation is useful, in fact you could have

done it by hand. The real usefulness of the code generator comes when, for example, complex objects like the following have to be parsed:

```
[
    {
        "id": 1,
        "roles": [
            {"id": 1, "type": "Author"},
            {"id": 2, "type": "Reviewer"}
        ],
        "data": {
            "params": [0, 3],
            "values": {
                "data": []
            },
            "keys": [13, 6, 7]
        },
        "extra": "none"
    }
]
```

Each object needs its dedicated model class plus a series of handlers for the lists. Doing a manual parsing is going to require a lot of boilerplate code and by consequence it will also be quite hard to maintain as time goes by. Thanks to code generation, an automated tool takes care of the most tedious part.

```
@JsonKey(required: true)
final int id;

@JsonKey(defaultValue: "no name")
final String name;
```

There are some extra annotations to instruct the behavior of the generator in certain situations. In the first case, the `required` marker throws an error if the key is not present in the JSON string. In the second case `defaultValue` puts the given value in the variable if the JSON key doesn't exist or if it's `null`.

15.1.2.1 Parsing lists

Parsing lists doesn't involve any extra effort because, again, the generator does all the work for you. Unless you're parsing a primitive type, such as strings or integers, you need a model class for the objects in the array.

```
{
    "total": 4,
    "values": ["A", "B", "C", "D"]
}
```

Declaring a `List<String>` instance variable is enough to tell `build_runner` to generate the needed code to parse a list of strings. In the next section we will show you what to do in case of lists of objects.

```
@JsonSerializable()
class Example {
    final int total;
    final List<String> values;
    Example(this.id, this.name);

    factory Example.fromJson(Map<String, dynamic> json)
        => _$ExampleFromJson(json);

    Map<String, dynamic> toJson()
        => _$ExampleToJson(this);
}
```

You could have put `JsonKey(defaultValue: [])` above `values` if you wanted it to be initialized with an empty array. Even if default values are not required, it's a good practice always defining them just to be sure that a variable is always initialized as expected.

15.1.2.2 Parsing nested objects

As you probably have already guessed, parsing nested object requires no efforts from your side because the implementation will be automatically generated. Let's take as example the same JSON string we have already seen in the previous section:

```
{
    "type": "Developers",
    "data": [
```

```
            { "id": 1, "name": "Alberto" },
            { "id": 2, "name": "Felix" },
            { "id": 3, "name": "Rémi" },
            { "id": 4, "name": "Matt" }
        ]
    }
```

We had already created a model class for the items in the array (`Person`, with the manual approach) but you could easily generate one with `build_runner`. What's missing is a model for the "outer" object, the one made up of `"data"` and `"type"`.

```
@JsonSerializable(explicitToJson: true)
class Data {
    final String type;
    final List<Person> data;
    Data(this.type, this.data);

    factory Data.fromJson(Map<String, dynamic> json)
        => _$DataFromJson(json);

    Map<String, dynamic> toJson()
        => _$DataToJson(this);
}
```

Use `explicitToJson: true` when you have a class that contains another class in one of its fields. In this case, in order to properly parse each `Person` object of the JSON array, you have to set `explicitToJson: true` on `Data` so that the inner-objects are included in the generation flow.

```
@JsonSerializable()
class Person { ... }

@JsonSerializable(explicitToJson: true)
class Data { ... }
```

Only the classes containing other classes need to have the `explicitToJson` parameter set. In case of primitive types, the annotation is not required. If your classes were big and deeply nested, the generator might take some more seconds but still it'd be faster than a manual approach.

ℹ The official documentation discourages the usage of the `part` keyword by the

developer [1]. The only case in which it's good is when code generation tools are involved, like in this case. For all the other needs, rely on a package-like structure using `library` and `export` statements.

15.2 Parsing XML

Another well-known data exchange format is XML which is still widely used even with the arrival of JSON, so you might be interested in knowing how to deal with it. One of the best packages available is called *xml* [2], made by Lukas Renggli.

```
<articles>
    <item>
        <name>Book</name>
        <quantity>5</quantity>
    </item>
    <item>
        <name>Tablet</name>
        <quantity>2</quantity>
    </item>
</articles>
```

Reading this string and converting it into a handy xml object is very easy as you just need to call `XmlDocument.parse()`. Version 4.1.0 and earlier of the package had a top-level method called `parse()` which is now deprecated.

```
import 'package:xml/xml.dart';

void main() {
    var xmlString = "<articles> ... </articles>";

    try {
        final xmlDoc = XmlDocument.parse(xmlString);
    } on XmlParserException {
        // error while parsing the string
    }
```

[1] https://dart.dev/guides/libraries/create-library-packages
[2] https://pub.dev/packages/xml

```
}
```

If the XML string is malformed, for example due to a syntax error, an exception is thrown. The `xmlDoc` variable is of type `XmlDocument` which provides many functionalities. It overrides the `toString()` so that the object can easily be printed.

```
print("$xmlDoc");
```

The returned string is "pretty" encoded, meaning that it's properly indented with spaces and tabulations. If you want to have more control on the output formatting, use `indent`:

```
final output = xmlDoc.toXmlString(
  pretty: true,
  indent: '-'
);

print(output);
```

Having set `pretty: true` the XML tree is not printed verbatim because useless leading spaces are removed and indentation is fixed. With `indent: '-'` the indentation spaces are replaced with - producing this:

```
<articles>
-<item>
--<name>Book</name>
--<quantity>5</quantity>
-</item>
-<item>
--<name>Tablet</name>
--<quantity>2</quantity>
-</item>
</articles>
```

By default the indentation uses a double white space `'  '` but you could also use `indent: '\t'` or anything else.

> ℹ When calling `XmlDocument.parse()` you need to pass a string in a proper UTF encoding. In Dart, `String` is a sequence of UTF-16 code units so everything works fine.
>
> ```
> // All good because strings in Dart are UTF encoded
> ```

```
final xmlDoc = XmlDocument.parse("<articles></articles>");
```

If you're retrieving the XML from the internet, for example via GET request, the string encoding might be different from UTF (it could be Latin1, for example). Since `parse()` works well only with UTF, you could have the need to use the `Latin1Decoder()` class.

```
// More info on GET requests and networking on chapter 17.
// This is a GET request which returns a 'Response' object
final response = await http.get("https://website.com/get/myFile.xml");

// Convert the response to a proper UTF format, which is how Dart
// strings are internally represented
final String xmlString = Latin1Decoder().convert(response.bodyBytes);

// You're now sure that parsing will execute successfully
final xmlDoc = XmlDocument.parse(xmlString);
```

The response body (`response.body`) is a `String` but it's not guaranteed to be in UTF encoding. If the response header specifies `charset=latin1` for example, then the string encoding will be Latin1. Thanks to `Latin1Decoder()` you can convert a latin1-encoded string into a Dart string (UTF-16).

```
AsciiDecoder().convert(response.bodyBytes);
Utf8Decoder().convert(response.bodyBytes);
```

They all work like `Latin1Decoder()`: use the `convert()` method to return a well-formed Dart string. This conversion should be taken into account not only when working with the xml package but in general (the same problem might happen when working with JSON as well).

15.2.1 Parsing strings

Once an XML-encoded string has been converted into a `XmlDocument`, there's the possibility to traverse the elements in many ways. Let's say we wanted to print the name of any article in the list:

```
final xmlString = """
    <articles>
        <item>
            <name>Book</name>
```

```
            <quantity>5</quantity>
        </item>
        <item>
            <name>Tablet</name>
            <quantity>2</quantity>
        </item>
    </articles>
""";

final xmlDoc = XmlDocument.parse(xmlString);

xmlDoc.findAllElements("name")
    .map((item) => item.text)
    .forEach(print);

// Prints:
// Book
// Tablet
```

Very intuitively, `findAllElements(String name)` returns an iterable object containing every XML element whose tag matches the given string. We could also have iterated on `item` and extracted the various values of the children by hand:

```
xmlDoc.findAllElements("item")
    .map((item) {
        final name = item.findElements("name").single.text;
        final qty = item.findElements("quantity").single.text;

        return "$name (amount = $qty)";
    })
    .forEach(print);

// Prints:
// Book (amount = 5)
// Tablet (amount = 2)
```

The `findElements()` method looks for any children of the currently selected node, it doesn't look on the entire tree. In order to obtain a reference to the node you can use:

- **single**: Checks if there's only one child node with the given name and returns it. If not,

an exception is thrown.

- **first**: Returns the first child node that matches the given name and returns it. If it's not present, an exception is thrown.
- **last**: Returns the last child node that matches the given name and returns it. If it's not present, an exception is thrown.

While `findAllElements()` finds **every** children in the tree with a given name, `findElements()` instead looks only for children of the **current** node. Look at this simple comparison to better understand the difference:

- `result` is an iterable containing 2 children, more precisely the two `<item>` tags representing the book and the tablet. With `findAllElements()` you're looking for any node across the entire tree.

```
final xmlDoc = XmlDocument.parse(xmlString);
final result = xmlDoc.findAllElements("item");

// <articles> is at the root and it contains 2 <item> nodes;
// findAllElements for any element whose tag is 'item'
```

- `result` is an iterable containing no children because it looks only for direct children of the current node. The current node is the root which has a single child (`<articles>`).

```
final xmlDoc = XmlDocument.parse(xmlString);
final result = xmlDoc.findElements("item");

// The current node is the root, which only contains
// <articles>; the various <item> elements are not direct
// children of the root
```

  When calling `parse()` the currently selected node becomes the root of the tree which only has the `<articles>` node. The root has no `<item>` children so the method finds nothing.

- In this case both methods return an iterable of size 1 which contains `<articles>` and all of its children.

```
final xmlDoc = XmlDocument.parse(xmlString);
final deepChildren = xmlDoc.findAllElements("articles");
final directChildren = xmlDoc.findElements("articles");
```

  `<articles>` is a direct children of the root, the currently selected node, so `findElements`

is able to get a result. At the same time, `<articles>` is the only tag in the tree so `findAllElements` will return an identical result.

In general you should use `findAllElements` when you want to get an entire list of elements and then inside it you call `findElements` to retrieve the specific values. This is an example of how they can be used together:

```
// Get every <item> node with 'findAllElements' and then
// use 'findElements' to find only direct children of the
// currently selected element.
final tot = xmlDoc.findAllElements("item")
    .map((item) =>
        int.parse(item.findElements("quantity").single.text)
    )
    .reduce((a, b) => a + b);

print("$tot"); // prints '7', the sum of all quantities
```

15.2.2 Building XML strings

Let's say we want to create an XML string representing a series of personal information such as name, surname and age. The final result has to look like this:

```
<people>
    <person>
        <name>Alberto</name>
        <surname>Miola</surname>
        <age>23</age>
    </person>
    <person>
        <name>Another</name>
        <surname>One</surname>
        <age>36</age>
    </person>
</people>
```

Thanks to the `class XmlBuilder` this job is quite easy but the code could get hard to read due to the big amount of nested functions. For this reason, it would be better creating series of methods for each recurring item, in this case the `<person>` child.

```
void addPerson(XmlDocument.XmlBuilder builder, {
    required String name,
    required String surname,
    required int age
}) {
    builder.element('person', nest: () {
        builder.element('name', nest: name);
        builder.element('surname', nest: surname);
        builder.element('age', nest: age);
    });
}
```

Each time `addPerson(...)` is called, a new `<person>` element is added and the code still remains readable. Using `nest` you can pass a primitive value, such a string/number, or a function which generally builds new XML items.

> Of course there is also the possibility to set attributes to specific elements. For example, if we wanted to add the `age` attribute we'd have to use `attribute()`:
>
> ```
> builder.element('person', nest: () {
>  builder.attribute("young", age <= 18);
>  // name, surname and age...
> });
> ```
>
> The result might look like `<person young="false"></person>` where `true` or `false` depends whether the age is lower or higher than 18.

Now you just need to create an instance of `XmlBuilder` and compose the string; once finished, call the `build()` method. Helper functions such as `addPerson` make the code more readable because nesting too many functions might be quite pesky.

```
final builder = XmlBuilder();

builder.processing('xml', 'version="1.0"');
builder.element('people', nest: () {
    addPerson(builder,
        name: "Alberto",
        surname: "Miola",
```

```
            age: 23);
        addPerson(builder,
            name: "Another",
            surname: "One",
            age: 23);
    });

    final peopleXML = builder.build();
```

Converting the XML object with `peopleXML.toXmlString(pretty: true);` is probably the best human-readable solution. However, you can also go for a plain `toString()` which uses no spaces nor indentation.